Introduction to Logic (CS) 2026

Lecture 1

Dan Frumin

University of Groningen

Tuesday 1 Sep 2026

Welcome to Introduction to Logic (CS)!

The team:



Dan Frumin



Helle Hvid Hansen



Gerard Renardel

Fundamental Computing group, CS, Bernoulli Institute, UG

Teaching Assistants

... all Computing Science BSc, MSc, and PhD students:

- Elitsa Todorova
- Kester Baur
- Huna Ruzafa
- Eliza Tchernavskikh
- Andy Tatman

They will be running tutorials and grading your work.
(Be nice to them :-))

General Motivation

What is logic?

- Reasoning is a typically human activity.
- Reasoning should be correct
- Logic is the study of inferences and reasoning, and judging it to be correct/incorrect

General Motivation

What is logic?

- Reasoning is a typically human activity.
- Reasoning should be correct
- Logic is the study of inferences and reasoning, and judging it to be correct/incorrect

Why study logic?

From Language, Proof and Logic, p.1:

> *. . . all rational inquiry depends on logic, on the ability of people to reason correctly most of the time, and, when they fail to reason correctly, on the ability of others to point out the gaps in their reasoning.*

What is logic?

Logic is the study of inferences and reasoning

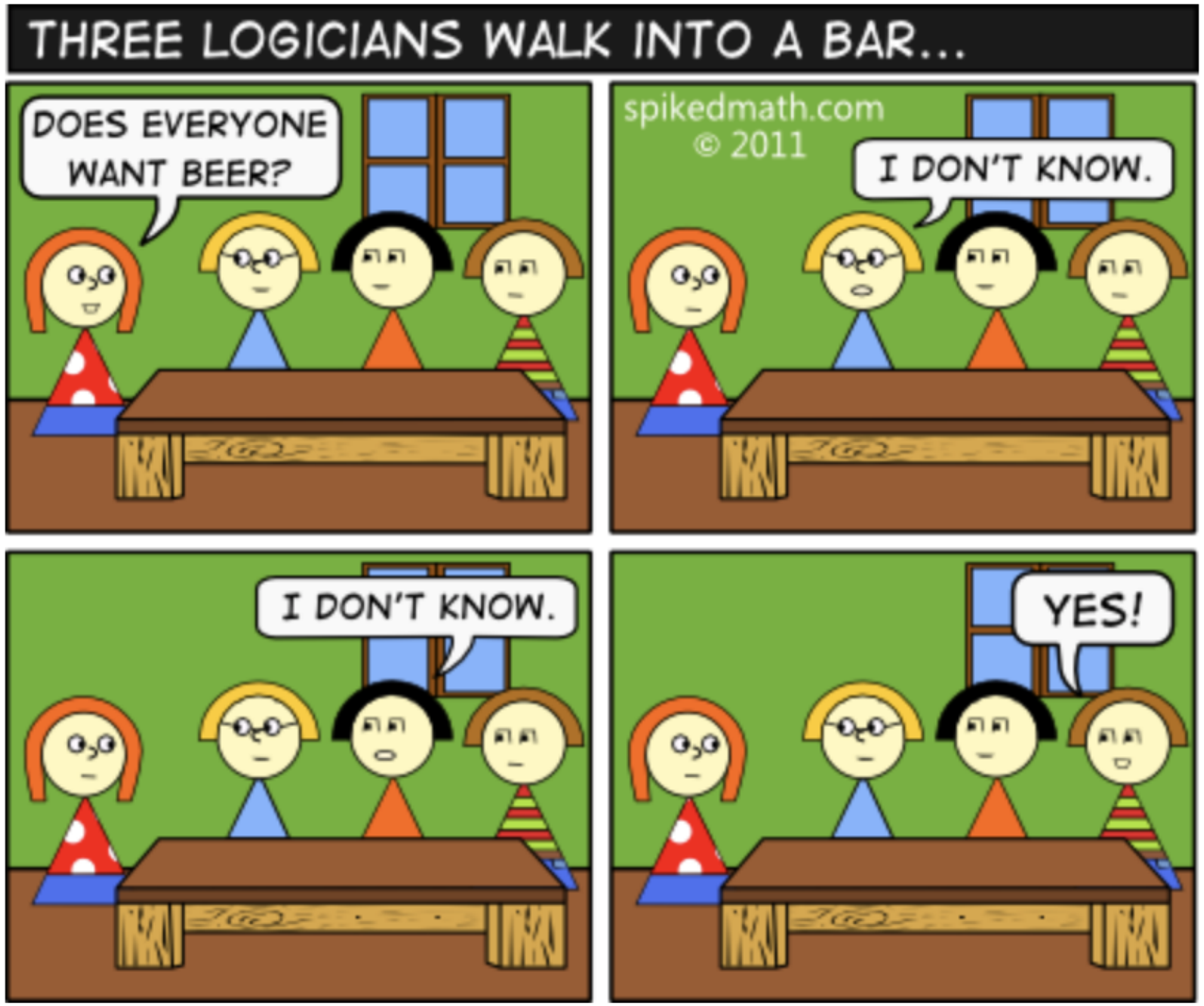
- Good, but too broad and too specific at the same time.
- Can you give your own definition of “logic”?

Logic as a Language of Reason

Logic is the study of inferences and reasoning

- Good, but broad. Can you give your own definition of “logic”?
- Reasoning in *abstract*
- *Formal* reasoning
- Reasoning *symobolically*
- Logic as an object: universal language for expressing reasoning

Logic as a Language of Information



Applications of Logic and Computer Science

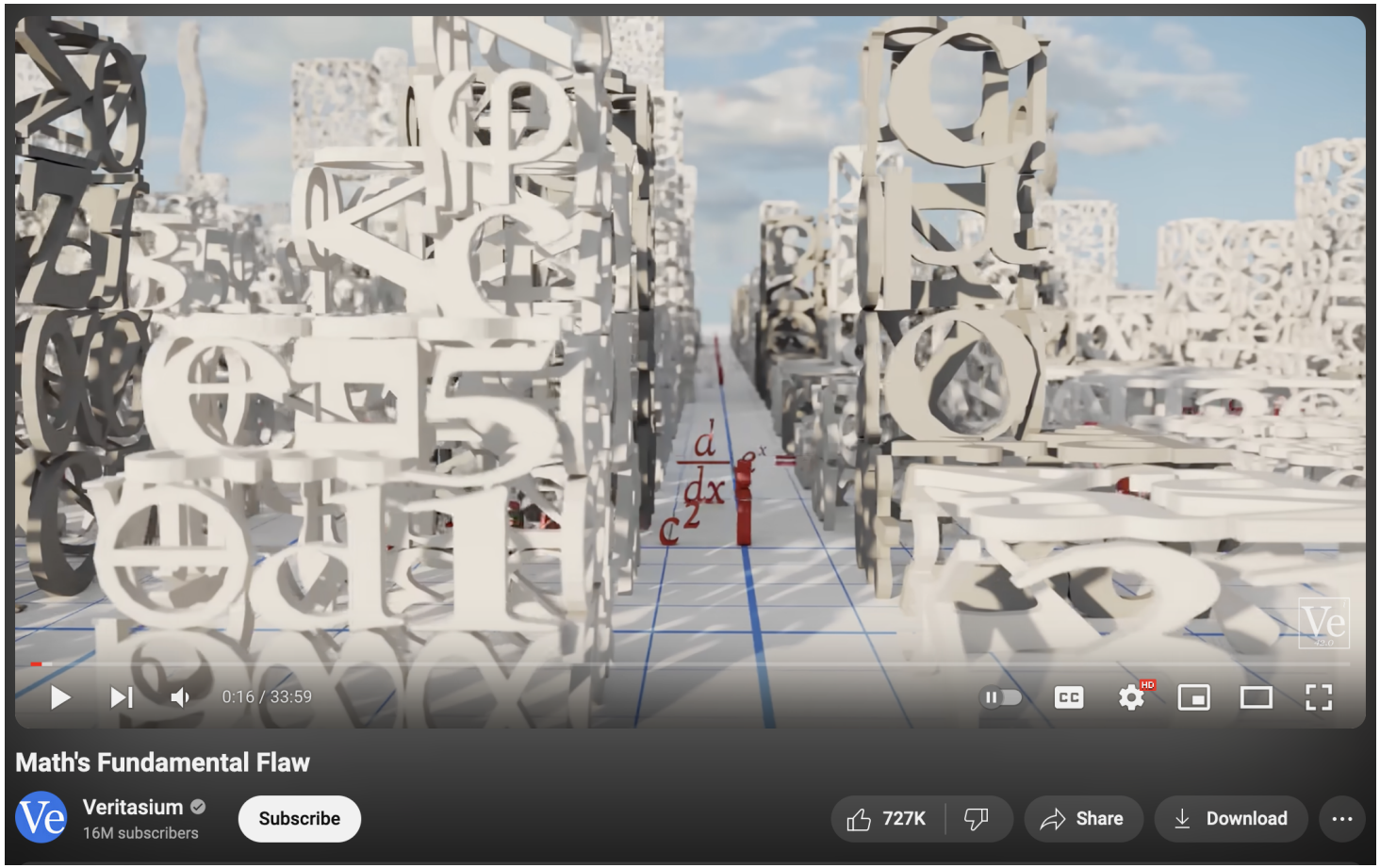
- **Hardware logic:** Bits and Boolean gates (cf. Computer Architecture)
- **Formal verification:** Reasoning about program behaviour (cf. Programming Fundamentals). Many different logics.
- **Theory of computation:** Reasoning about computability and computational resources (cf. Languages and Machines, Computational Complexity)
- And much more: type systems, λ-calculus and Turing machines, abstract interpretation, Hoare logic, model checking (cf. Models and Semantics of Computation, Programs and Interactive Proofs, Modal Logic and Proof Theory)...

Applications of Logic and Computer Science

- **Hardware logic:** Bits and Boolean gates (cf. Computer Architecture)
- **Formal verification:** Reasoning about program behaviour (cf. Programming Fundamentals). Many different logics.
- **Theory of computation:** Reasoning about computability and computational resources (cf. Languages and Machines, Computational Complexity)
- And much more: type systems, λ-calculus and Turing machines, abstract interpretation, Hoare logic, model checking (cf. Models and Semantics of Computation, Programs and Interactive Proofs, Modal Logic and Proof Theory)...

Even moreso, the quest for logical foundations of mathematics gave birth to computers.

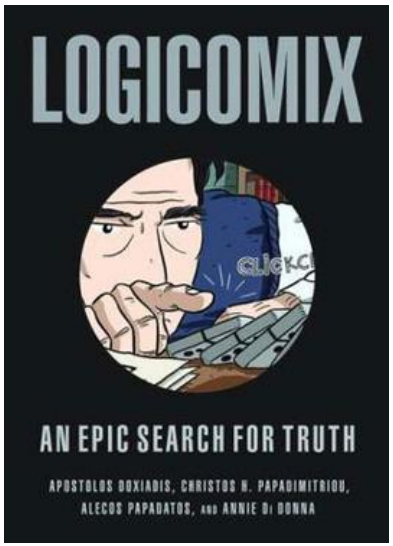
Logic and Computer Science



Logic and Computing Science

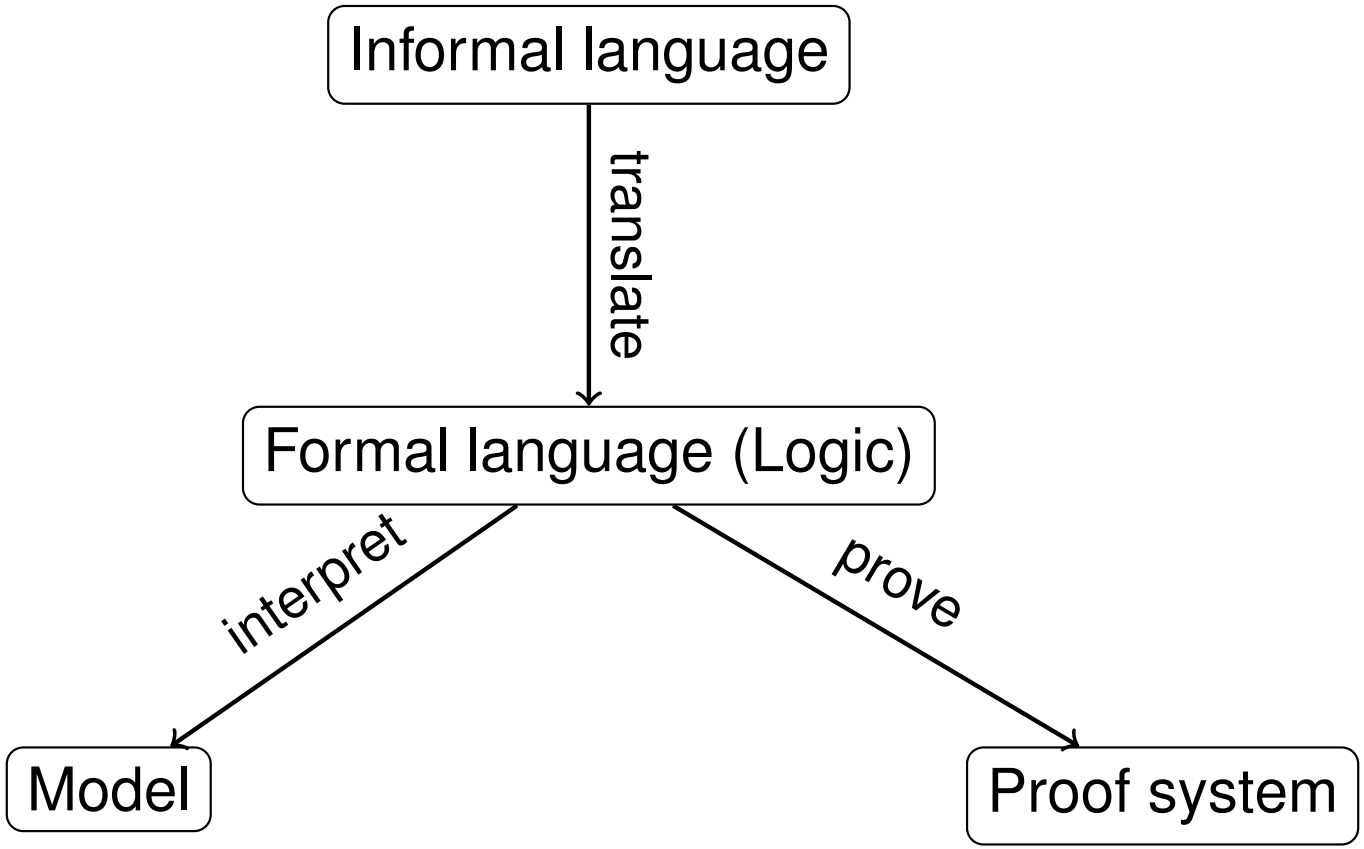
Want to know more about the foundations of mathematics and the birth of CS?

- Veritasium, *"Math's Fundamental Flaw"*.
- Prof. Moshe Vardi, *"From Aristotle to the iPhone"* (World Logic Day 2021 lecture).



A. Doxiadis and C.H. Papadimitriou,
"Logicomix: An epic search for truth"

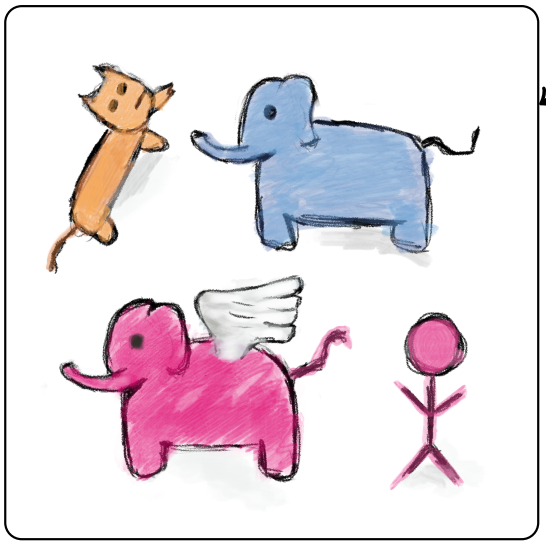
Logic in this course

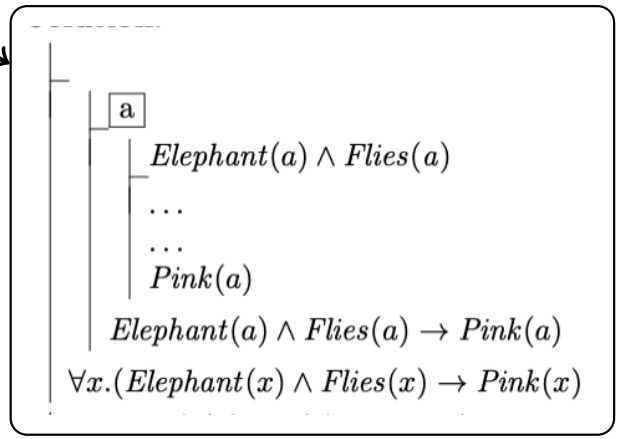


Logic in this course

All flying elephants are pink

$\forall x(Elephant(x) \wedge Flies(x) \rightarrow Pink(x))$





Today's Lecture

1. Course Organisation
2. Validity of Arguments
3. Atomic Sentences in FOL
4. Formal Proofs with Atomic Sentences

Today's Lecture

1 Course Organisation
2 Validity of Arguments
3 Atomic Sentences in FOL
4 Formal Proofs with Atomic Sentences

About this course

This course provides a first introduction to:

- The formal, symbolic languages of *first-order logic* (FOL) and *propositional logic* (PL).
- The semantics/interpretation of FOL and PL.
- The notion of formal proof, and formal proofs for FOL and PL

About this course

This course provides a first introduction to:

- The formal, symbolic languages of *first-order logic* (FOL) and *propositional logic* (PL).
- The semantics/interpretation of FOL and PL.
- The notion of formal proof, and formal proofs for FOL and PL

Learning Goals: reasoning *with mathematical precision*

1. *translate* English sentences into PL and FOL
2. examine the *validity* of arguments
3. develop *formal proofs* for inferences
4. transform logical sentences to *normal form(s)*
5. apply the *Horn algorithm* to Horn sentences

Course setup

All information on course organisation is found in Brightspace.
Make sure to read it all! Only a few remarks here.

- Lectures: Mon 17-19, Tue 11-13
- Tutorials: Wed/Thu depending on the mentor grp (except for the first week)
- Be alert to Brightspace announcements and requests to fill in forms etc.
- Assessment: two homeworks, a midterm, an exam
- Homework and midterm grades *can only improve* your final grade, those are formative assessments

Course materials and resources

- Brightspace: All course info. All communication. Grades. Turn on notifications!
- Textbook: *Language, Proof and Logic* (LPL) by Plummer, Barwise, Etchemendy, 2011, 2nd ed. (1st ed. OK).
- Reader/syllabus is complementary to LPL.
- Software: FitchVIZIER

Assessment

Assessment components:

- Two homework assignments (HW1, HW2)
- Midterm test (M)
- Exam (E)

Let HWME = (8∗HW1+16∗HW2+16∗M+60∗E)/100.
The final grade G will be determined by:

- First exam attempt: if E $\geq$ 5.0 then G = max(E,HWME) else G = E.
- Second exam attempt: G = E.

(Exam grade must be at least 5.0. HW and Midterm can only improve your grade.)

In general, G $\geq$ 6.0 is needed to pass. (Note: 5.5 rounds to 6.0 and 5.4 rounds to 5.0.)

Overview of the course

week 1: Valid and sound arguments. Atomic sentences and their proofs

week 2: Boolean connectives $\neg$, $\wedge$ and $\vee$; proof strategy and tactics (HW1 posted)

week 3: Conditionals $\rightarrow$ and $\leftrightarrow$ and their proof rules (HW1 due)

week 4: Truth tables, tautologies, normal forms

week 5: Quantifiers $\forall$, $\exists$ and their proof rules (Midterm on Mon)

week 6: Normal forms for quantifiers

week 7: Set theory; first-order models (HW2 posted)

week 8: Skolemization, Horn sentences (HW2 due)

week 10: Exam

week 14: Resit

Today's Lecture

1 Course Organisation

2 Validity of Arguments

3 Atomic Sentences in FOL

4 Formal Proofs with Atomic Sentences

Learning goals this week

Topics:

- Valid and sound arguments.
- How to build atomic sentences in first-order logic (FOL).
- Rules for constructing formal proofs of atomic sentences.
- Get started with FitchVIZIER

At the end of this week, you are able to:

- Write informal arguments in Fitch format.
- Analyse the validity and soundness of arguments.
- Express simple English statements as atomic sentences in FOL.
- Construct simple formal proofs for atomic sentences.

What is an argument?

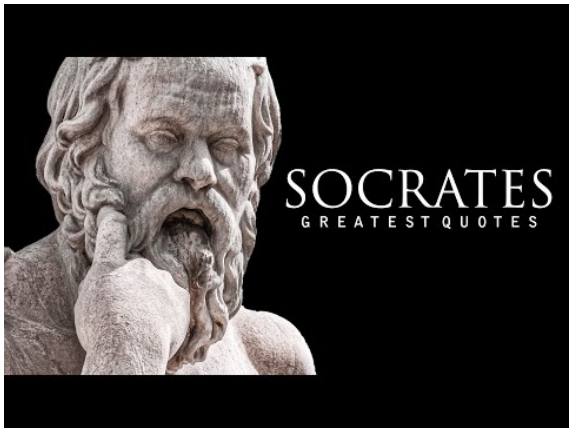
If I miss the train, then I will be late.
I miss the train.
∴ I will be late.

What is an argument?

If I miss the train, then I will be late.	*Premise*
I miss the train.	*Premise*
⊢ I will be late.	*Conclusion*

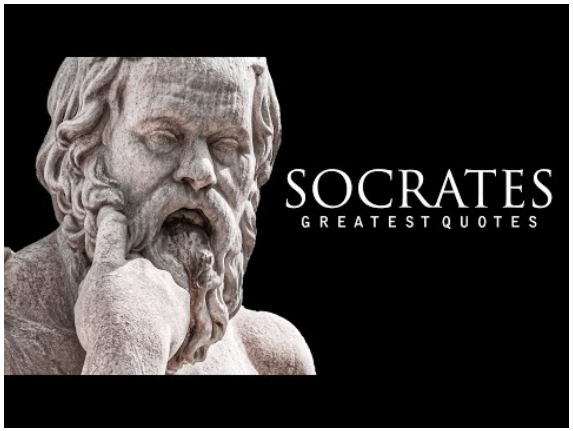
Some other arguments

All men are mortal.
Socrates is a man.
∴ Socrates is mortal.



Some other arguments

All men are mortal.
Socrates is a man.
∴ Socrates is mortal.



It will rain or it will storm.
If it will rain, I stay home.
If it will storm, I stay home.
∴ I stay home.



Inference schemes: patterns of reasoning

We only look at the form of the argument. For example:

Inference schemes: patterns of reasoning

We only look at the form of the argument. For example:

All men are mortal.
Socrates is a man.
∴ Socrates is mortal.

All dogs are animals.
Scruffy is a dog.
∴ Scruffy is an animal.

Inference schemes: patterns of reasoning

We only look at the form of the argument. For example:

All men are mortal.
Socrates is a man.
∴ Socrates is mortal.

All dogs are animals.
Scruffy is a dog.
∴ Scruffy is an animal.

It will rain or it will storm.
If it will rain, I stay home.
If it will storm, I stay home.
∴ I stay home.

I will stay or I will go.
If I stay, there will be trouble.
If I go, there will be trouble.
∴ There will be trouble.

Inference schemes: patterns of reasoning

We only look at the form of the argument. For example:

All men are mortal.
Socrates is a man.
∴ Socrates is mortal.

All dogs are animals.
Scruffy is a dog.
∴ Scruffy is an animal.

All Ps are Qs.
x is a P.
∴ x is a Q.

It will rain or it will storm.
If it will rain, I stay home.
If it will storm, I stay home.
∴ I stay home.

I will stay or I will go.
If I stay, there will be trouble.
If I go, there will be trouble.
∴ There will be trouble.

P or Q.
If P, then R.
If Q, then R.
∴ R

A different argument

If you are a member, then you get a discount.
You are not a member.

You do not get a discount.

A different argument

If you are a member, then you get a discount.
You are not a member.
∴ You do not get a discount.

Inference scheme:

If P then Q.
It is not the case that P.
∴ It is not the case that Q.

$P \to Q$
$\neg P$
∴ $\neg Q$

The definition of validity

From Language, Proof and Logic, p. 44:

> *An argument is valid if the conclusion must be true in any circumstance in which the premises are true.*
> *We say that the conclusion of a logically valid argument is a logical consequence of its premises.*
>
> *An argument is sound if it is valid and the premises are all true.*

Logic focuses on **validity** rather than soundness.

Demonstrating validity: Proof

Question: How to show that an argument is valid?

Demonstrating validity: Proof

Question: How to show that an argument is valid?
Answer: Give a proof!

Demonstrating validity: Proof

Question: How to show that an argument is valid?
Answer: Give a proof!

From Language, Proof and Logic p. 46-47:

> *A proof is a step-by-step demonstration that a conclusion (say S) follows from some premises (say P, Q, R).*

- Each step establishes an intermediate conclusion which is an "obvious consequence" of the premises and the other intermediate conclusions established so far.
- The proof ends when S is established in this way.

Example of an informal proof

1. All animals are mortal
2. All humans are animals
3. Socrates is a human

---

4. Socrates is an animal (from 2, 3)
5. Socrates is mortal (from 1, 4)

Example of a formal proof

1.	$P \vee (Q \wedge R)$	
2.	P	
3.	$P \vee Q$	$\vee$ Intro: 2
4.	$P \vee R$	$\vee$ Intro: 2
5.	$(P \vee Q) \wedge (P \vee R)$	$\wedge$ Intro: 3, 4
6.	$Q \wedge R$	
7.	Q	$\wedge$ Elim: 6
8.	R	$\wedge$ Elim: 6
9.	$P \vee Q$	$\vee$ Intro: 7
10.	$P \vee R$	$\vee$ Intro: 8
11.	$(P \vee Q) \wedge (P \vee R)$	$\wedge$ Intro: 9, 10
12.	$(P \vee Q) \wedge (P \vee R)$	$\vee$ Elim: 1, 2–5, 6–11

Demonstrating non-validity: Counterexample

Question: How to show that an argument is *not valid*?

Demonstrating non-validity: Counterexample

Question: How to show that an argument is *not valid*?
Answer: Give a counterexample!

Demonstrating non-validity: Counterexample

Question: How to show that an argument is *not valid*?
Answer: Give a counterexample!

From Language, Proof and Logic, p. 63:

> *To show that a sentence Q is not a consequence of premises P_1 , . . . , P_n, we must ... demonstrate that it is possible for P_1 , . . . , P_n to be true while Q is simultaneously false. That is, we must show that there is a possible situation or circumstance in which the premises are all true while the conclusion is false. Such a circumstance is said to be a counterexample to the argument.*

Back to the example...

If P, then Q
It is not the case that P

It is not the case that Q

Is this valid? (Poll: Yes/No)

Back to the example...

If P, then Q
It is not the case that P

It is not the case that Q

Is this valid? (Poll: Yes/No)

Take for example:
P: You are in Amsterdam.
Q: You are in the Netherlands.

Back to the example...

If P, then Q
It is not the case that P

It is not the case that Q

Is this valid? (Poll: Yes/No)

Take for example:
P: You are in Amsterdam.
Q: You are in the Netherlands.

Then we have: $P \rightarrow Q$ and $\neg P$ are true, but $\neg Q$ is false.

This situation is a valid counterexample, so the argument is not valid!

Today's Lecture

1 Course Organisation

2 Validity of Arguments

3 Atomic Sentences in FOL

4 Formal Proofs with Atomic Sentences

The language of first-order logic (FOL)

...is built from:

- **Atomic sentences**
- Boolean connectives (and, or, not)
- Conditionals (if-then, if-and-only-if)
- Quantifiers (for all ..., there exists ...)

Atomic Sentences

- A *sentence* is a statement that is True or False.
- An *atomic sentence* is a sentence that cannot be broken down into other sentences.
- E.g. *"Socrates is the teacher of Plato, and Plato is a philosopher"* is a sentence, but it is not atomic.

Atomic Sentences

- A *sentence* is a statement that is True or False.
- An *atomic sentence* is a sentence that cannot be broken down into other sentences.
- E.g. *"Socrates is the teacher of Plato, and Plato is a philosopher"* is a sentence, but it is not atomic.

Atomic sentences can be combined using

- Boolean connectives (and, or, not)
- Conditionals (if-then, if-and-only-if)
- Quantifiers (for all ..., there exists ...)

Atomic Sentences

- A *sentence* is a statement that is True or False.
- An *atomic sentence* is a sentence that cannot be broken down into other sentences.
- E.g. *"Socrates is the teacher of Plato, and Plato is a philosopher"* is a sentence, but it is not atomic.

Atomic sentences are built from basic building blocks:

- **constants** (to give names to things/objects, e.g. *plato*, *socrates*)
- **functions** (to map things to other things, e.g. *teacher*(*plato*))
- **predicates** (to express properties of and relations between things, e.g. *Philosopher*(*plato*))
- **=** (the equality predicate, e.g. *teacher*(*plato*) $=$ *socrates*)

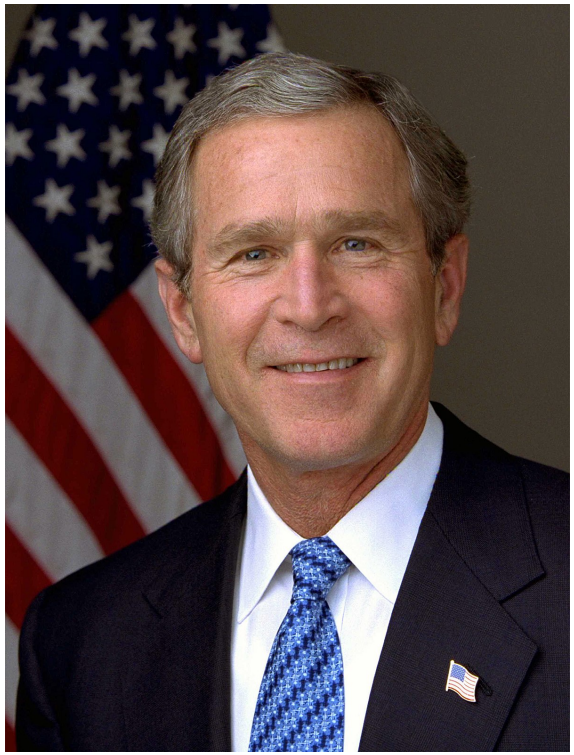
Names in Logic: Unambiguous

President George Bush served two terms in the White House.
True or false?

Names in Logic: Unambiguous

President George Bush served two terms in the White House.
True or false? Depends on who you mean...





not allowed: A name without a thing

not allowed: A name without a thing



404 Not Found

Not Found

The requested URL /oldpage.html was not found on this server.

Apache/2.0.54 (Fedora) Server at www.example.com Port 80

Done

```
(base) Helles-UG-MBP:~ hhhansen$ pyton
-bash: pyton: command not found
(base) Helles-UG-MBP:~ hhhansen$ pythonn
-bash: pythonn: command not found
(base) Helles-UG-MBP:~ hhhansen$
```

not allowed: A name without a thing



404 Not Found

Not Found

The requested URL /oldpage.html was not found on this server.

Apache/2.0.54 (Fedora) Server at www.example.com Port 80

Done

```
(base) Helles-UG-MBP:~ hhhansen$ pyton
-bash: pyton: command not found
(base) Helles-UG-MBP:~ hhhansen$ pythonn
-bash: pythonn: command not found
(base) Helles-UG-MBP:~ hhhansen$
```

Which *things* are there? Depends on the chosen domain of discourse, the collection of things we want to reason about.

Constants in first-order logic

- Constants play the role of names in first-order logic.
- Constants/names are unambiguous references to things/objects.

Constants in first-order logic

- Constants play the role of names in first-order logic.
- Constants/names are unambiguous references to things/objects.

OK Something with one name.
OK Something with two names.
OK Something without a name.
Wrong One name for two things.
Wrong A name without a thing.

Constants in first-order logic

- Constants play the role of names in first-order logic.
- Constants/names are unambiguous references to things/objects.

OK	Something with one name.
OK	Something with two names.
OK	Something without a name.
Wrong	One name for two things.
Wrong	A name without a thing.

- Constants are written in lower-case.
  Examples: a, b, c, max, claire, ...
  Use abbreviations/translation key:

gbsr : George Bush Sr. *socrates* : Socrates
gbjr : George Bush Jr. *plato* : Plato

Terms and functions symbols in first-order logic

- Constants are the simplest kind of terms in FOL.
- Adding function symbols to language of FOL allow us to build complex terms:

Terms and functions symbols in first-order logic

- Constants are the simplest kind of terms in FOL.
- Adding function symbols to language of FOL allow us to build complex terms:
  - father(claire) is a term: the father of Claire.
  - mother(max) is a term: the mother of Max.

Terms and functions symbols in first-order logic

- Constants are the simplest kind of terms in FOL.
- Adding function symbols to language of FOL allow us to build complex terms:
  - father(claire) is a term: the father of Claire.
  - mother(max) is a term: the mother of Max.
  - $2 + 3$ is a term: the sum of 2 and 3 (+ takes two inputs).
  - $\min_4(4, 1, 7, 99)$ is a term: the minimum of . . . (takes four inputs)

Terms and functions symbols in first-order logic

- Constants are the simplest kind of terms in FOL.
- Adding function symbols to language of FOL allow us to build complex terms:
  - father(claire) is a term: the father of Claire.
  - mother(max) is a term: the mother of Max.
  - $2 + 3$ is a term: the sum of 2 and 3 (+ takes two inputs).
  - $\min_4(4, 1, 7, 99)$ is a term: the minimum of . . . (takes four inputs)
- Functions symbols can be nested to build even more complex terms:

$$\text{father}(\text{mother}(\text{max})),$$
$$1 + \min_4(2 + 3, 5, 7, 8)$$
$$\ldots$$

Function symbols represent functions

- Let D be the domain of discourse.
- Function symbols represent functions on D:

$$\text{father} : D \to D \qquad + : D \times D \to D \qquad \min_4 : D^4 \to D$$

(cf. function names in programming)

- Functions have a fixed arity, which is the number of inputs it applies to.
- In fact: Constants can be seen as 0-ary function symbols.

Function symbols represent functions

- Let D be the domain of discourse.
- Function symbols represent functions on D:

$$\text{father} : D \to D \qquad + : D \times D \to D \qquad \min_4 : D^4 \to D$$

  (cf. function names in programming)
- Functions have a fixed arity, which is the number of inputs it applies to.
- In fact: Constants can be seen as 0-ary function symbols.

Inputs to functions are often called arguments (cf. programming).

Function symbols represent functions

- Let D be the domain of discourse.
- Function symbols represent functions on D:

$$\text{father} : D \to D \qquad + : D \times D \to D \qquad \min_4 : D^4 \to D$$

  (cf. function names in programming)
- Functions have a fixed arity, which is the number of inputs it applies to.
- In fact: Constants can be seen as 0-ary function symbols.

Inputs to functions are often called arguments (cf. programming).

Beware! 'argument' has two different meanings in logic:

- series of statements in which the conclusion is supposed to follow from the others;
- an object to which a function (or predicate) is applied.

Predicate symbols (aka relation symbols)

From Language, Proof and Logic, p. 20:

> *Predicate symbols are symbols used to express some property of objects or some relation between objects.*

Every predicate has an *arity*: the number of terms it applies to.

Predicate symbols (aka relation symbols)

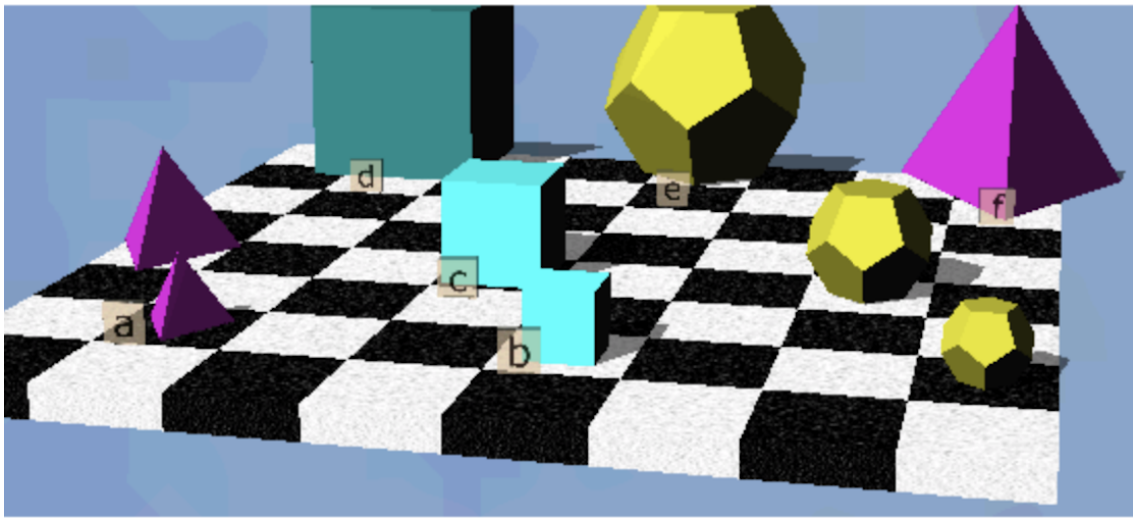
From Language, Proof and Logic, p. 20:

> *Predicate symbols are symbols used to express some property of objects or some relation between objects.*

Every predicate has an *arity*: the number of terms it applies to.

Properties are unary predicates, having one argument.
Examples of unary predicates:

- Cube(b): b is a cube
- Dodec(b): b is a dodecahedron
- Small(b): b is small
- Blue(b): b is blue



Binary predicates

Binary predicates have two arguments.

- SameShape(a,b): a has the same shape as b
- Larger(a,b): a is larger than b
- SameCol(a,b): a is in the same column as b.
- LeftOf(a,b): a is located nearer to the left edge of the grid than b.

Binary predicates

Binary predicates have two arguments.

- SameShape(a,b): a has the same shape as b
- Larger(a,b): a is larger than b
- SameCol(a,b): a is in the same column as b.
- LeftOf(a,b): a is located nearer to the left edge of the grid than b.

Other examples:

- Love(romeo, juliet): to love someone
- $5 > 3$: being greater than
- Knows(max,claire): to know someone
- $3 \mid 12$: being divisible by

n-ary predicates ($n > 2$)

n-ary predicates have n arguments. (n-ary relation symbols)

- Between(a,b,c): a, b, and c are in the same row, column, or diagonal and a is between b and c.
- x likes y more than z
- x gives y to z for w
- x hears from y that z loves w more than v

n-ary predicates ($n > 2$)

n-ary predicates have n arguments. (n-ary relation symbols)

- Between(a,b,c): a, b, and c are in the same row, column, or diagonal and a is between b and c.
- x likes y more than z LikesMoreThan(x,y,z)
- x gives y to z for w
- x hears from y that z loves w more than v

n-ary predicates ($n > 2$)

n-ary predicates have n arguments. (n-ary relation symbols)

- Between(a,b,c): a, b, and c are in the same row, column, or diagonal and a is between b and c.
- x likes y more than z LikesMoreThan(x,y,z)
- x gives y to z for w GivesToFor(x,y,z,w)
- x hears from y that z loves w more than v

n-ary predicates ($n > 2$)

n-ary predicates have n arguments. (n-ary relation symbols)

- Between(a,b,c): a, b, and c are in the same row, column, or diagonal and a is between b and c.
- x likes y more than z LikesMoreThan(x,y,z)
- x gives y to z for w GivesToFor(x,y,z,w)
- x hears from y that z loves w more than v
  HearsFromThatLovesMoreThan(x,y,z,w,v) or just H(x,y,z,w,v)

The identity predicate =

- A binary predicate with a fixed interpretation:

  a = b says that a and b refer to the same object.

- Identity can be used between terms, e.g.:

  father(max) = father(mother(claire))

The identity predicate =

- A binary predicate with a fixed interpretation:

  a = b says that a and b refer to the same object.

- Identity can be used between terms, e.g.:

  father(max) = father(mother(claire))

Note the difference between symbols (syntax) and their interpretation (semantics):

- 'a' and 'b' are clearly not the same symbol, but they may have the same interpretation (refer to the same object)
- 'father(max)' and 'father(mother(claire))' are different as terms, but they may refer to the same person

How to translate ’is’ into FOL?



1. Don Diego de la Vega is Zorro.
2. Don Diego de la Vega is a nobleman.

How to translate 'is' into FOL?



1. Don Diego de la Vega is Zorro.
2. Don Diego de la Vega is a nobleman.

1. d = z, with the translation key

   d: Don Diego de la Vega;
   z: Zorro

How to translate 'is' into FOL?



1. Don Diego de la Vega is Zorro.
2. Don Diego de la Vega is a nobleman.

1. d = z, with the translation key

   d: Don Diego de la Vega;
   z: Zorro

2. N(d), with the translation key

   N(x): x is a nobleman;
   d: Don Diego de la Vega

Notation for predicates in FOL

In first-order logic, predicates are usually:

- represented by capitalised words or letters: Blue, Larger, S, ...
- or in PascalCase: SameShape, GivesAt, ...
- written in prefix-notation: Blue(a), Larger(c,b), S(3), ...

Exception: some well-known binary predicates are

- represented by common symbols: $=$, $<$, $>$, $\leq$, $\geq$
- written in infix-notation:
  - a = b
  - $2 < 5$
  - $3 > 5$
  - $5 \leq 10$
  - $7 \geq 7$

Order of arguments

The following sentences mean something completely different:

- Loves(romeo,juliet)
- Loves(juliet,romeo)

Idem for the following two sentences:

- $2 < 4$
- $4 < 2$

What about: a = b and b = a ?

Atomic sentences in first-order logic (FOL)

Constructing atomic sentences:

- An atomic sentence is obtained by applying a predicate to (a correct number of!) terms.
- Atomic sentences evaluate to True or False, depending on the interpretation of terms and predicates.
- Recall: terms are built from constants and function symbols

Examples:

- $2 < 5$
- $x < 5$
- $a+2 \geq \min(3+\min(7,b+6),c)$
- Older(father(max),mother(claire))

Atomic sentences in first-order logic (FOL): Exercise

Exercise: Which of the following are atomic sentences?

- Larger(a,b)
- Knows(father(claire),max)
- father(father(claire)) = max
- $2 + 7 < 42$
- PrimeNumber(3+4)
- Dodec(a,b)
- Cube(Blue(a))
- father(claire)
- father(Knows(claire,max))
- $2 + (3 < 7)$

Atomic sentences in first-order logic (FOL): Exercise

Exercise: Which of the following are atomic sentences?

- Larger(a,b) ✓
- Knows(father(claire),max) ✓
- father(father(claire)) = max ✓
- $2 + 7 < 42$ ✓
- PrimeNumber(3+4) ✓
- Dodec(a,b) ✗
- Cube(Blue(a)) ✗
- father(claire) ✗
- father(Knows(claire,max)) ✗
- $2 + (3 < 7)$ ✗

FOL is not a single language

Guideline from Language, Proof and Logic, p. 29:

> *Usually, the overall goal is to come up with a language that can say everything you want, but that uses the smallest "vocabulary" possible. Picking the right names and predicates is the key to doing this.*

We design a specific FOL vocabulary for each problem context.

FOL vocabulary examples

FOL vocabulary examples

Tarski's World (blocks on chessboard):

- Constants: a,b,c,d, ...
- Functions: leftmost, rightmost
- Properties: Blue, Red, Cube, Dodec, Small, Large, ...
- Binary predicates: SameShape, SameSize, Larger,...

FOL vocabulary examples

Tarski's World (blocks on chessboard):

- Constants: a,b,c,d, ...
- Functions: leftmost, rightmost
- Properties: Blue, Red, Cube, Dodec, Small, Large, ...
- Binary predicates: SameShape, SameSize, Larger,...

Romeo and Juliet:

- Constants: romeo, juliet, mercutio, tybalt, ladycapulet, ...
- Functions: father, mother, ...
- Properties: Montague, Capulet, Female, Male, ...
- Binary predicates: Loves, Kills, Marries, ...

Languages of FOL and Propositional Logic (PL)

$$\text{Propositional Logic} \left\{ \begin{array}{l} \text{Atomic sentences} \\ \text{Boolean connectives} \\ \text{Conditionals} \\ \text{Quantifiers} \end{array} \right\} \text{FOL}$$

Atomic sentences

- ...**in FOL**, are constructed from predicate symbols and terms.

$$B(a), R(c, d), Q(b, f, e, d, a), Loves(romeo, juliet), a = b, 2 < 3, ...$$

- ...**in PL**, have no inner structure. They are represented by single letters (often capitals):

$$P, Q, R, S, ...$$

Today's Lecture

1 Course Organisation

2 Validity of Arguments

3 Atomic Sentences in FOL

4 Formal Proofs with Atomic Sentences

To show validity of an argument, give a proof!

From Language, Proof and Logic p. 46-47:

> *A proof is a step-by-step demonstration that a conclusion follows from some premises...*

To show validity of an argument, give a proof!

From Language, Proof and Logic p. 46-47:

> *A proof is a step-by-step demonstration that a conclusion follows from some premises...*

Some arguments are immediately valid:

No human is a daffodil.

No human is a daffodil.

Zorro is the same person as Zorro.

Others require multiple steps:

All animals are mortal
All humans are animals
Socrates is a human

Socrates is an animal
Socrates is mortal

Deductive Systems and Formal Proofs

What is a deductive system?

- A deductive system is a collection of inference rules in a formal language (such as FOL) that are predetermined as being valid/correct. **(Principles of reasoning)**
- A system called $\mathcal{F}$ for *natural deduction* is developed in the book.
- The tool Fitch is based on $\mathcal{F}$, but it is slightly more flexible.

Deductive Systems and Formal Proofs

What is a deductive system?

- A deductive system is a collection of inference rules in a formal language (such as FOL) that are predetermined as being valid/correct. **(Principles of reasoning)**
- A system called $\mathcal{F}$ for *natural deduction* is developed in the book.
- The tool Fitch is based on $\mathcal{F}$, but it is slightly more flexible.

What is a formal proof?

- A formal proof is defined w.r.t. to a deductive system.
- A formal proof is a sequence of inference steps (in the same formal language as the deductive system) where each step is justified by the inference rules of the deductive system.

Recall: Example of a formal proof

1.	$P \vee (Q \wedge R)$	
2.	P	
3.	$P \vee Q$	$\vee$ Intro: 2
4.	$P \vee R$	$\vee$ Intro: 2
5.	$(P \vee Q) \wedge (P \vee R)$	$\wedge$ Intro: 3, 4
6.	$Q \wedge R$	
7.	Q	$\wedge$ Elim: 6
8.	R	$\wedge$ Elim: 6
9.	$P \vee Q$	$\vee$ Intro: 7
10.	$P \vee R$	$\vee$ Intro: 8
11.	$(P \vee Q) \wedge (P \vee R)$	$\wedge$ Intro: 9, 10
12.	$(P \vee Q) \wedge (P \vee R)$	$\vee$ Elim: 1, 2–5, 6–11

General shape of proofs in system $\mathcal{F}$

1.	P_1	(Premise)
2.	P_2	(Premise)
3.	P_3	(Premise)
4.	T_1	Justification
5.	T_2	Justification
⋮		⋮
k.	C	Justification

...possibly with nested subproofs.

- Rules: introduction and elimination of logic constructs; reiteration.
- Justification: how a sentence follows from earlier ones by applying the rules.
- **Today's rules:** =Introduction, =Elimination, Reiteration

= Introduction (system $\mathcal{F}$ rule)

Idea: Every object is identical to itself.
Example: Zorro is the same person as Zorro.

I.e., for all terms (incl. names) t, we have that $t = t$. Identity is reflexive.

= Introduction (system $\mathcal{F}$ rule)

Idea: Every object is identical to itself.
Example: Zorro is the same person as Zorro.

I.e., for all terms (incl. names) t, we have that $t = t$. Identity is reflexive.

General form:

$$\begin{array}{|ll} \vdots & \\ k. & t = t \quad =\text{Intro} \\ \vdots & \end{array}$$

Examples:

$$\begin{array}{|lll} \vdots & & \\ 5. & \mathit{zorro} = \mathit{zorro} & =\text{Intro} \\ 6. & \mathit{father}(\mathit{claire}) = \mathit{father}(\mathit{claire}) & =\text{Intro} \\ \vdots & & \end{array}$$

$=$ Elimination (system $\mathcal{F}$ rule)

Idea:
For all terms s and t, if $s = t$ then you may replace any occurrence of s with t in any sentence.

= Elimination (system $\mathcal{F}$ rule)

Idea:
For all terms s and t, if $s = t$ then you may replace any occurrence of s with t in any sentence.

Examples:

Consider	constant	ddlv:	Don Diego de la Vega
	constant	adlv:	Don Alejandro de la Vega
	constant	zorro:	Zorro
	unary predicate	C(x):	x is from California

1. ddlv = zorro
2. C(ddlv)

3. C(zorro) =Elim: 2, 1

= Elimination (system $\mathcal{F}$ rule)

Idea:
For all terms s and t, if $s = t$ then you may replace any occurrence of s with t in any sentence.

Examples:

Consider	constant	ddlv:	Don Diego de la Vega
	constant	adlv:	Don Alejandro de la Vega
	constant	zorro:	Zorro
	unary predicate	C(x):	x is from California

1. ddlv = zorro
2. C(ddlv)

---

3. C(zorro) =Elim: 2, 1

1. C(adlv)
2. adlv = father(ddlv)
3. ddlv = zorro

---

4. C(father(ddlv)) =Elim: 1, 2
5. C(father(zorro)) =Elim: 4, 3

= Elimination (system $\mathcal{F}$ rule)

Idea:
For all terms s and t, if $s = t$ then you may replace any occurrence of s with t in any sentence.

General form:

⋮		
i.	$A(s)$	Justification (or premise)
⋮		
j.	$s = t$	Justification (or premise)
⋮		
k.	$A(t)$	= Elim: i, j
⋮		

Note:

- $A(s)$ is a pattern. It denotes a sentence in which s occurs.
- The order in which $A(s)$ and $s = t$ occur above the application of =Elim does not matter.
- The order of the references i,j in =Elim: i,j does matter! First reference to the pattern $A(s)$. Then reference to identity $s = t$.

Example: Proof of transitivity of =

Identity is transitive:

For all terms s, t, u: if $s = t$ and $t = u$ then $s = u$.

Example: Proof of transitivity of =

Identity is transitive:

For all terms s, t, u: if $s = t$ and $t = u$ then $s = u$.

Proof in system $\mathcal{F}$:

1.	$s = t$		
2.	$t = u$		
3.	$s = u$	=Elim: 1, 2	*(in line 1, replace t with u)*

Note again the order of references in the justification:
First the pattern, then the identity.
Why is =Elim: 2, 1 incorrect?

Example: Proof of symmetry of =

Identity is symmetric:

For all terms s and t:

if $s = t$
then $t = s$.

In Fitch form:

$s = t$

$t = s$

Example: Proof of symmetry of =

Identity is symmetric:

For all terms s and t:

if $s = t$
then $t = s$.

In Fitch form:

$s = t$

$t = s$

Proof in system $\mathcal{F}$:

1. $s = t$		
2. $s = s$	=Intro	
3. $t = s$	=Elim: 2, 1	*(in line 2, replace the left s with t)*

Note: the order of justification references:
First the pattern, then the identity.
In line 3: = Elim: 1, 2 would not be correct (why?).

Properties of identity

We have formally proved in system $\mathcal{F}$ that the identity relation = has the properties:

reflexivity: For all t: $t = t$

symmetry: For all s, t: if $s = t$ then $t = s$

transitivity: For all s, t, u: if $s = t$ and $t = u$ then $s = u$

A problem?

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

A problem?

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

Maybe try:

1. $P(a)$
2. $b = a$

---

3. $P(b)$ =Elim: 1, 2 **??**

A problem?

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

Maybe try:

1. $P(a)$
2. $b = a$

---

3. $P(b)$ =Elim: 1, 2 **??**

Not allowed in $\mathcal{F}$: $b = a$ only allows us to replace b by a, not the other way round.

Solution

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

Solution:

1. $P(a)$
2. $b = a$

---

3. $b = b$ =Intro

Solution

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

Solution:

1. $P(a)$
2. $b = a$

---

3. $b = b$ =Intro
4. $a = b$ =Elim: 3, 2

Solution

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

Solution:

1. $P(a)$
2. $b = a$

---

3. $b = b$ =Intro
4. $a = b$ =Elim: 3, 2
5. $P(b)$ =Elim: 1, 4

Solution

? How to prove:

1. $P(a)$
2. $b = a$

---

3. $P(b)$

Solution:

1. $P(a)$
2. $b = a$

---

3. $b = b$ =Intro
4. $a = b$ =Elim: 3, 2
5. $P(b)$ =Elim: 1, 4

Prove an instance of symmetry on the way.

Replacing one or more occurrences

1.	$Q(a, a, b, b)$	
2.	$a = c$	
3.	$b = d$	
4.	$Q(c, a, b, b)$	=Elim: 1, 2
5.	$Q(c, a, d, d)$	=Elim: 4, 3

Replacing one or more occurrences

1. $Q(a, a, b, b)$	
2. $a = c$	
3. $b = d$	
4. $Q(c, a, b, b)$	=Elim: 1, 2
5. $Q(c, a, d, d)$	=Elim: 4, 3

In step 4, one occurrence of a is replaced by c

In step 5, two occurrences of b are replaced by d

System $\mathcal{F}$ rules: Reiteration

Evidently,

No human is a daffodil.

...

No human is a daffodil.

System $\mathcal{F}$ rules: Reiteration

Evidently,

> No human is a daffodil.
> ⊢
> ...
> No human is a daffodil.

Reiteration rule allows to restate sentences that have already been proved:

⋮		
j.	A	Justification (or premise)
⋮		
k.	A	Reit: j
⋮		

When proofs get very long, the Reiteration rule can be convenient.

How to find a formal proof?

Like playing chess:

- strict rules,
- tactics and strategies can help (more next week).



Wrapping up

Learning goals: At the end of this week, you are able to

- Write arguments in Fitch format.
- Analyse the validity and soundness of arguments.
- Express simple English statements as atomic sentences in FOL.
- Construct simple formal proofs for atomic sentences.

For readings and exercises this week, see Brightspace.
Remember to go through all information in Brightspace!

Week 1 tutorials: TAs will help with

- getting started with software (FitchVIZIER).
- exercises and questions on Week 1 material

Prepare well to make the best use of tutorials.